

```
43 printf("\n\n");
44
45 printf("Page Replacement Order:\n");
46
47 // Process each page
48 for (int i = 0; i < n; i++)
49 {
50     int page = referenceString[i];
51     int pageFound = 0;
52
53     // Check whether page is already present
54     for (int j = 0; j < MAX_FRAMES; j++)
55     {
56         if (frames[j] == page)
57         {
58             pageFound = 1;
59             break;
60         }
61     }
62 }
```

Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page Replacement Order:

Page 7 -> 7 - -

Page 0 -> 7 0 -

Page 1 -> 7 0 1

Page 2 -> 2 0 1

Page 3 -> 2 3 1

Page 0 -> 2 3 0

Page 4 -> 4 3 0

Page 2 -> 4 2 0

Page 3 -> 4 2 3

Page 0 -> 0 2 3

Total Page Faults = 10

```

1  #include <stdio.h>
2
3  #define MAX_FRAMES 3
4
5  // Function to print frames
6  void printFrames(int frames[], int n)
7  {
8      for (int i = 0; i < n; i++)
9      {
10         if (frames[i] == -1)
11             printf(" - ");
12         else
13             printf(" %d ", frames[i]);
14     }
15
16     printf("\n");
17 }
18
19 int main()
20 {
21     int frames[MAX_FRAMES];
22
23     // Initialize frames
24     for (int i = 0; i < MAX_FRAMES; i++)
25     {
26         frames[i] = -1;
27     }
28
29     int pageFaults = 0;
30
31     int referenceString[] =
32     {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};

```

sources Compile Log Debug Find Results Console Close

- Errors: 0
- Warnings: 0

```

C:\Users\SRIRAM\OneDrive\D
+ v
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page Replacement Order:
Page 7 -> 7 - -
Page 0 -> 7 0 -
Page 1 -> 7 0 1
Page 2 -> 2 0 1
Page 3 -> 2 0 3
Page 4 -> 2 4 3
Page 0 -> 2 0 3

Total Page Faults: 7

-----
Process exited after 0.1655 seconds with return value 0
Press any key to continue . . . |

```

main.c

Run

Output

```
1 #include <stdio.h>
2
3 int main() {
4     int base, limit, logicalAddress, physicalAddress;
5
6     // Input Base and Limit registers
7     printf("Enter Base Register value: ");
8     scanf("%d", &base);
9
10    printf("Enter Limit Register value: ");
11    scanf("%d", &limit);
12
13    // Input Logical Address
14    printf("Enter Logical Address: ");
15    scanf("%d", &logicalAddress);
16
17    // Check validity of logical address
```

Enter Base Register value: 1000
Enter Limit Register value: 500
Enter Logical Address: 250
Valid Logical Address
Physical Address = 1250

=== Code Execution Successful ===

```
// Prefer an empty frame
for (int j = 0; j < MAX_FRAMES; j++)
{
    if (frames[j] == -1)
    {
        lruPage = j;
        break;
    }

    if (usageHistory[j] > usageHistory[lruPage])
    {
        lruPage = j;
    }
}

int replacedPage = frames[lruPage];

// Replace page
frames[lruPage] = page;
usageHistory[lruPage] = 0;

if (replacedPage != -1)
{
    printf("Replace %d with %d: ",
           replacedPage, page);
}
else
{
    printf("Load into an empty frame: ");
}

printFrames(frames, MAX_FRAMES);
```

Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page Replacement Order:

Page 7 -> Load into an empty frame: 7 - -

Page 0 -> Load into an empty frame: 7 0 -

Page 1 -> Load into an empty frame: 7 0 1

Page 2 -> Replace 7 with 2: 2 0 1

Page 3 -> Replace 2 with 3: 3 0 1

Page 4 -> Replace 1 with 4: 3 0 4

Page 2 -> Replace 3 with 2: 2 0 4

Page 3 -> Replace 2 with 3: 3 0 4

Page 2 -> Replace 4 with 2: 3 0 2

Total Page Faults: 9

Process exited after 0.197 seconds with return value 0
Press any key to continue . . . |

```

scanf("%d", &recordNumber);

if (recordNumber == 0)
{
    break;
}

rewind(file);

int found = 0;

while (fread(&record, sizeof(struct Record), 1, file))
{
    if (record.recordNumber == recordNumber)
    {
        printf("Record Number: %d\n", record.recordNumber);
        printf("Data: %s\n", record.data);
        found = 1;
        break;
    }
}

if (!found)
{
    printf("Record not found.\n");
}

fclose(file);

```

C:\Users\SRIRAM\OneDrive\D × + ▾

```

Enter records (Enter 0 as record number to exit):
Record Number: 389
Data: sriram
Record Number: 234
Data: sdferf
Record Number: 322
Data: vgdvsavr
Record Number: 213
Data: dfdhrsth
Record Number: 1322
Data: tgrht
Record Number: sgrtye5y|

```

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int pageTable[100];
6     int n, pageSize, logicalAddress;
7     int pageNumber, offset, frameNumber, physicalAddress;
8     int i;
9
10    printf("Enter number of pages: ");
11    scanf("%d", &n);
12
13    printf("Enter page size: ");
14    scanf("%d", &pageSize);
15
16    printf("\nEnter page table entries:\n");
17    for(i = 0; i < n; i++)
18    {
19        printf("Frame number for Page %d: ", i);
20        scanf("%d", &pageTable[i]);
21    }
22 }
```

Enter number of pages: 4

Enter page size: 100

Enter page table entries:

Frame number for Page 0: 5

Frame number for Page 1: 2

Frame number for Page 2: 8

Frame number for Page 3: 0

Enter logical address: 250

--- Page Table Translation ---

Logical Address = 250

Page Number = 2

Offset = 50

Page 2 maps to Frame 8

Physical Address = 850

=== Code Execution Successful ===

```
{  
    scanf("%d", &limit);  
  
    printf("Enter Logical Address: ");  
    scanf("%d", &logicalAddress);  
  
    /* Check memory protection */  
    if(logicalAddress >= 0 && logicalAddress < limit)  
    {  
        physicalAddress = base + logicalAddress;  
  
        printf("\nAccess Granted!\n");  
        printf("Logical Address = %d\n", logicalAddress);  
        printf("Physical Address = %d\n", physicalAddress);  
    }  
}
```

Enter Base Register value: 1000
Enter Limit Register value: 500
Enter Logical Address: 250

Access Granted!
Logical Address = 250
Physical Address = 1250

=== Code Execution Successful ===


```
4  {
5  printf("Enter offset: ");
26 scanf("%d", &offset);
27
28 if(pageNumber < 0 || pageNumber >= pages ||
29    offset < 0 || offset >= pageSize)
30 {
31     printf("\nInvalid Logical Address!\n");
32 }
33 else
34 {
35     frameNumber = pageTable[pageNumber];
36
37     physicalAddress = frameNumber * pageSize + offset;
38
39     printf("\nPage Number = %d\n", pageNumber);
40     printf("Frame Number = %d\n", frameNumber);
41     printf("Offset = %d\n", offset);
42     printf("Physical Address = %d\n", physicalAddress);
43 }
```

Enter number of pages: 4

Enter page size: 100

Enter frame number for each page:

Page 0 -> Frame: 5

Page 1 -> Frame: 2

Page 2 -> Frame: 8

Page 3 -> Frame: 1

Enter Page Number: 2

Enter Offset: 50

Page Number = 2

Frame Number = 8

Offset = 50

Physical Address = 850

=== Code Execution Successful ===

lab 26 11.cpp × lab 27.cpp × lab 28.cpp × lab 29.cpp × lab 30.cpp

```

7   printf("Inside the thread\n");
8   pthread_exit(NULL);
9 }
10
11 void fun()
12 {
13     pthread_t ptid;
14
15     pthread_create(&ptid, NULL, func, NULL);
16
17     printf("This line may be printed before thread terminates\n");
18
19     if (pthread_equal(ptid, pthread_self()))
20     {
21         printf("Threads are equal\n");
22     }
23     else
24     {
25         printf("Threads are not equal\n");
26     }
27
28     pthread_join(ptid, NULL);
29
30     printf("This line will be printed after thread ends\n");
31 }
32
33 int main()
34 {
35     fun();
36
37     return 0;
38 }

```

C:\Users\SRIRAM\OneDrive\D × + v

This line may be printed before thread terminates
 Inside the thread
 Threads are not equal
 This line will be printed after thread ends

 Process exited after 0.2246 seconds with return value 0
 Press any key to continue . . . |

Resources Compile Log Debug Find Results Console Close

Message

```
4 {  
5 // Effective Memory Access Time With TLB  
6  
7     ematWithTLB =  
8         hitRatio * (tlbAccessTime + memoryAccessTime) +  
9         missRatio * (tlbAccessTime + 2 * memoryAccessTime);  
10  
11     printf("\n--- TLB Performance Analysis ---\n");  
12     printf("Memory Access Time = %.2f ns\n", memoryAccessTime);  
13     printf("TLB Access Time = %.2f ns\n", tlbAccessTime);  
14     printf("TLB Hit Ratio = %.2f\n", hitRatio);  
15     printf("TLB Miss Ratio = %.2f\n", missRatio);  
16  
17     printf("\nEffective Memory Access Time Without TLB = %.2f ns\n",  
18         ematWithoutTLB);  
19  
20     printf("Effective Memory Access Time With TLB = %.2f ns\n",  
21         ematWithTLB);  
22 }
```

Enter Memory Access Time (in ns): 100

Enter TLB Access Time (in ns): 20

Enter TLB Hit Ratio (0 to 1): 0.8

--- TLB Performance Analysis ---

Memory Access Time = 100.00 ns

TLB Access Time = 20.00 ns

TLB Hit Ratio = 0.80

TLB Miss Ratio = 0.20

Effective Memory Access Time Without TLB = 200.00 ns

Effective Memory Access Time With TLB = 140.00 ns

Improvement = 60.00 ns faster

=== Code Execution Successful ===